

Tổng Hợp Kiến Thức

1. Giới thiệu Sequelize và ORM

- **ORM (Object-Relational Mapping):** Lớp trung gian giữa code JavaScript và Database. Giúp thao tác DB bằng JavaScript thay vì SQL thuần.
- **Sequelize:** ORM phổ biến cho Node.js, hỗ trợ nhiều DB (MySQL, PostgreSQL, SQLite, v.v.).

2. Định nghĩa Models

- `models/category.js` :

```
'use strict';
const { Model, DataTypes } = require('sequelize');

module.exports = (sequelize) => {
  class Category extends Model {
    static associate(models) { /* Quan hệ ở đây */ }
  }
  Category.init({
    name: { type: DataTypes.STRING, unique: true, allowNull: false },
    description: DataTypes.TEXT
  }, {
    sequelize,
    modelName: 'Category',
    tableName: 'Categories', // Tên bảng DB
    timestamps: true // createdAt, updatedAt
  });
  return Category;
};
```

- **DataTypes phổ biến:** `STRING` , `TEXT` , `INTEGER` , `DECIMAL` , `DATE` , `BOOLEAN` .
- Thêm Models vào `models/index.js` :

```
db.Category = require('./category')(sequelize, Sequelize);
db.Product = require('./product')(sequelize, Sequelize);
// ...
Object.keys(db).forEach(modelName => {
  if (db[modelName].associate) {
    db[modelName].associate(db);
  }
});
```

3. CRUD cơ bản

- `routes/categoryRoutes.js` :

```
const express = require('express');
const router = express.Router();
const { Category } = require('../models');

// Lấy tất cả
router.get('/', async (req, res) => {
  const categories = await Category.findAll();
  res.json(categories);
});
```

```
// Lấy theo ID
router.get('/:id', async (req, res) => {
  const category = await Category.findByPk(req.params.id);
  if (!category) return res.status(404).json({ message: 'Không tìm thấy' });
  res.json(category);
});

// Tạo mới
router.post('/', async (req, res) => {
  const newCategory = await Category.create(req.body);
  res.status(201).json(newCategory);
});

// Cập nhật
router.put('/:id', async (req, res) => {
  const [updatedRows] = await Category.update(req.body, { where: { id: req.params.id } });
  if (updatedRows === 0) return res.status(404).json({ message: 'Không tìm thấy để cập nhật' });
  const updatedCategory = await Category.findByPk(req.params.id);
  res.json(updatedCategory);
});

// Xóa
router.delete('/:id', async (req, res) => {
  const deletedRows = await Category.destroy({ where: { id: req.params.id } });
  if (deletedRows === 0) return res.status(404).json({ message: 'Không tìm thấy để xóa' });
  res.status(204).send();
});

module.exports = router;
```

4. Quan hệ giữa các bảng

- **Một - Nhiều (One-to-Many):**

- `models/category.js` :

```
Category.hasMany(Product, { foreignKey: 'categoryId', as: 'products' });
```

- `models/product.js` :

```
Product.belongsTo(Category, { foreignKey: 'categoryId', as: 'category' });
```

- **Eager Loading:**

```
const categories = await Category.findAll({
  include: [{ model: Product, as: 'products' }]
});
```

- **Một - Một (One-to-One):**

- `models/user.js` :

```
User.hasOne(Profile, { foreignKey: 'userId', as: 'profile' });
```

- `models/profile.js` :

```
Profile.belongsTo(User, { foreignKey: 'userId', as: 'user' });
```

- **Eager Loading:**

```
const users = await User.findAll({
  include: [{ model: Profile, as: 'profile' }]
});
```

- **Nhiều - Nhiều (Many-to-Many):**

- `models/product.js` :

```
Product.belongsToMany(Tag, { through: ProductTag, foreignKey: 'productId', otherKey: 'tagId', as: 'tags' })
```

- `models/tag.js` :

```
Tag.belongsToMany(Product, { through: ProductTag, foreignKey: 'tagId', otherKey: 'productId', as: 'products' })
```

- **Eager Loading:**

```
const products = await Product.findAll({
  include: [{ model: Tag, as: 'tags' }]
});
```

5. Truy vấn nâng cao

- **where** (Điều kiện lọc):

```
const { Op } = require('sequelize');
const products = await Product.findAll({
  where: {
    price: { [Op.gte]: 500 }, // giá >= 500
    name: { [Op.like]: '%Laptop%' }, // tên chứa "Laptop"
    categoryId: { [Op.in]: [1, 2] } // categoryId thuộc [1, 2]
  }
});
```

- **Toán tử Op phổ biến:** `eq` , `ne` , `gt` , `gte` , `lt` , `lte` , `in` , `notIn` , `like` , `iLike` , `startsWith` , `endsWith` , `substring` , `regexp` , `notRegexp` , `and` , `or` .

- **order** (Sắp xếp):

```
const products = await Product.findAll({
  order: [
    ['price', 'DESC'], // giảm dần theo giá
    ['name', 'ASC'] // tăng dần theo tên (sau khi sắp xếp theo giá)
  ]
});
```

- **limit** và **offset** (Phân trang):

```
const pageNumber = 2;
const pageSize = 10;
const offset = (pageNumber - 1) * pageSize;

const products = await Product.findAll({
  limit: pageSize,
  offset: offset
});
```